

Writeup CTF : Transformation

Plateforme : picoCTF 2021 | Catégorie : Reverse Engineering | Difficulté : Facile

Objectif / Description

Déchiffrer une chaîne de caractères (ressemblant à du texte asiatique) accompagnée d'un snippet Python de chiffrement d'une ligne. Le but est de comprendre la logique de transformation binaire pour coder le script inverse.

Outils & Concepts clés

- Manipulation d'encodage et tables de caractères (ASCII / Unicode).
- Opérations bit-à-bit (Bitwise Shift `<<`, `>>`).
- Fonctions Python natives : `ord()` pour obtenir la valeur décimale et `chr()` pour reconvertir en caractère.
- Mathématiques modulaires (Division entière `//`, Modulo `%`).

Démarche pas à pas

1. Reconnaissance & Analyse Statique

Le challenge nous fournit une chaîne chiffrée (豔捌罢規𐄂形梲獐楮獐𐄂樓潦𐄂形寔坵捡𐄂) et un algorithme d'encodage Python compacté :

```
''.join([chr((ord(flag[i]) << 8) + ord(flag[i + 1])) for i in range(0, len(flag), 2)])
```

L'analyse de ce code révèle que le programme prend les caractères du flag original deux par deux (`range(0, len(flag), 2)`). Il décale le premier caractère de 8 bits vers la gauche (`<< 8` , ce qui équivaut mathématiquement à multiplier par 256) et lui additionne la valeur du second caractère. Deux caractères de 8 bits sont ainsi "empaquetés" en un seul nombre de 16 bits.

Le piège de la traduction

L'ordinateur interprète ces blocs de 16 bits résultants comme des caractères dans la table Unicode, ce qui correspond par hasard à la plage des sinogrammes (CJK). Essayer de traduire le texte via Google Translate est un "rabbit hole" classique ; il s'agit d'un pur artefact mathématique, pas de stéganographie linguistique.

2. Exploitation & Création du Script Inverse

Pour inverser le processus, il faut lire la valeur décimale du caractère Unicode avec `ord()`, puis extraire les deux octets d'origine. La division entière par 256 (ou un décalage binaire `>> 8`) permet de récupérer le premier caractère (quotient), et le modulo 256 (ou un masque binaire `& 0xFF`) permet de récupérer le second (reste).

Script d'exploitation développé (le décodeur 16-bits) :

```
chaine_encodee = '灑捌罢覩𐄂𐄂形梲獍楮獾𐄂𐄂搜潦𐄂形寔坵捡似'
resultat_clair = ""

for char in chaine_encodee:
    valeur = ord(char)

    # Extraction du 1er caractère (Division entière par 256)
    char1 = valeur // 256

    # Extraction du 2eme caractère (Modulo 256)
    char2 = valeur % 256

    # Reconversion en texte et concaténation
    resultat_clair += chr(char1) + chr(char2)

print(resultat_clair)
```

3. Obtention du flag

L'exécution de notre script de décodage révèle le flag attendu :

```
picoCTF{16_bits_inst34d_of_8_b7f62ca5}
```

Leçons retenues & Automatismes à garder

- **Capitalisation des outils** : Ce script de "dépaquetage" binaire 16-bits est à archiver dans sa boîte à outils (ex: `decode_16bit_unicode.py`), car la manipulation d'encodage est récurrente en CTF.

• **Connaître ses équivalences mathématiques et binaires :**

- Un décalage à gauche (`<< 8`) est une multiplication par 256.
- Un décalage à droite (`>> 8`) est une division entière par 256 (`// 256`).
- Un masque binaire (`& 0xFF`) est l'équivalent du modulo 256 (`% 256`).